



Security Assessment Final Report



Lido Staking Router

June 2026

Prepared for Lido

Table of Contents

Project Summary.....	4
Project Scope.....	4
Project Overview.....	4
Protocol Overview.....	4
Assessment Methodology.....	6
Threat and Security Overview.....	7
Findings Summary.....	9
Severity Matrix.....	9
Detailed Findings.....	10
Critical Severity Issues.....	13
C-01: DoS of Accounting Oracle report through direct donation.....	13
Medium Severity Issues.....	14
M-01: Possible revert in sanity checks due to `_lastVaultBalanceAfterTransfer` value set during migration.....	14
M-02: Non-Lido Consolidation Can Grief Lido Validator Selection.....	16
M-03: Potential loss of deposits in fresh deploy.....	17
M-04: CL balance increase can mistakenly be flagged as decrease, causing revert.....	18
M-05: Decrease sanity check does not account for deposits which allows deposit value to be dropped from accounting.....	20
Low Severity Issues.....	21
L-01: TWG can be DoSed due to ability to trigger the same withdrawals multiple times.....	21
L-02: Potential to top up a validator that is not yet deposited.....	22
L-03: `_calculateTotalProtocolFeeShares` does not use post-rebase share rate.....	23
L-04: Deposits and Top ups that use nearly the full deposit reserve and unreserved ETH can be grieved. 24	
L-05: Potentially unsafe `maxPositiveTokenRebase` parameter supplied to sanity checker.....	26
L-06: Rebase smoothing is malleable.....	27
L-07: APR safety cap calculation does not take into account activated balance.....	28
Informational Severity Issues.....	29
I-01: `DepositSecurityModule.canDeposit()` should reflect emergency setting quota larger than guardians.length.....	29
I-02: Consolidations should check deposit pause edge case.....	30
I-03: TopupGateway validator check is inefficient and does not require sorting.....	31
I-04: Inconsistencies in rate limit mechanisms.....	32
I-05: Missing return value description for `depositedForCurrentReport`.....	33
I-06: Incorrect reference to `obtainDepositData` function.....	34
I-07: consolidationEthAmountPerDayLimit discrepancy in docs.....	35
I-08: Incorrect reference to variable `clPendingValidatorsBalance`.....	36
I-09: Incorrect calculation in docs for `clRewards`.....	37

I-10: Incorrect comment for state variable.....	38
Disclaimer.....	39
About Certora.....	39

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Lido Staking Router	https://github.com/lidofinance/core	Initial: b0c3278d Fix Review: af095e48	EVM

Project Overview

This document describes the security review of **Lido Staking Router**. The work was undertaken from **March 17th, 2026** to **June 29th, 2026**.

The following contract list is included in our scope:

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Lido is a liquid staking protocol that takes user ETH, stakes it through a network of node operators, and mints stETH as the user's liquid claim on that staked position. From a security perspective, the protocol is coordinating a large amount of value across several moving parts: the ETH buffer, validator deposits, oracle-reported balances, reward distribution, withdrawal finalization, and module-level accounting.

Today, Lido's core flow is as follows: Users deposit ETH, the protocol routes it into validators, oracle reports update the system's view of consensus-layer balances, and withdrawals are processed through the withdrawal queue. The main audit focus is to determine that funds flow correctly, whether accounting stays consistent across reports, and whether node operators or privileged actors can manipulate deposits, exits, or withdrawals.

This update expands the staking router into a more flexible coordination layer for newer validator mechanics. The router now supports Ox02 withdrawal-credential modules, balance-based accounting, larger validator balances, and more nuanced deposit allocation across modules.

This update also includes a consolidation flow so that existing Ox01 validators can be converted into Ox02 validators, and so other validators can consolidate their balances to certain Lido-owned validators.

There is a new top up flow included in this scope, in addition to the existing deposit flow. Instead of only creating new 32 ETH validators, Lido can add ETH to existing eligible Ox02 validators. The TopUpGateway verifies beacon-chain proof data, calculates per-validator top-up limits, and then routes the operation through StakingRouter, which pulls depositable ETH from Lido and submits the deposits.

The triggerable withdrawals gateway adds another sensitive control surface. It lets authorized actors trigger full validator withdrawals through the newer execution-layer mechanism, with fees, rate limits, pause controls, and notifications back into the staking modules.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Assets

- Staked ETH (user deposits) and CL/EL rewards
- Withdrawal Vault ETH balance
- Lido validator balances (pre-deposit 1 ETH, full 32 ETH, top-ups, consolidations)
- Accounting oracle report integrity (CL pending balance, vault balance snapshots)
- TWG / Consolidation / Exit rate-limit capacity
- Deposit security quorum

Actors

- Stakers / Users
- DAO / Admin (sets limits, pauses deposits)
- Accounting Oracle (off-chain reporter)
- Deposit Security Module guardians
- Node Operators / Lido validators
- External validators (non-Lido withdrawal credentials)
- TWG / Consolidation / Top-Up Gateway callers
- Attacker (rich enough to fund/slash a non-Lido validator)

Trust Assumptions

- CL pending balance only grows from Lido-initiated deposits
- Withdrawal vault balance at migration reflects no in-flight withdrawals
- Consolidations originate from Lido-controlled withdrawal credentials
- Top-ups target only activated validators

- Oracle report sanity bounds match real validator lifecycle
- Off-chain code correctly filters non-Lido-sourced consolidations
- Deposit pause via oversized quorum is honored everywhere deposits are gated
- Rate-limit replenishment semantics are consistent across gateways

Attack Vectors

- Griefing the oracle via unsolicited ETH deposits to Lido validators (sanity-check revert / DoS)
- Migration-time snapshot mismatch causing CL-balance-decrease revert
- Cross-validator consolidation griefing using non-Lido source + self-slash to cancel
- Bypass of post-upgrade report check when lastProcessingRefSlot == 0, risking lost deposits
- Replay/duplicate triggerable withdrawals to exhaust TWG limit (DoS)
- Top-up routed to a not-yet-activated pre-deposit validator
- Stale clPendingBalance post-migration causing accounting drift
- Emergency deposit pause (quorum > guardians) not reflected in canDeposit / consolidation preconditions
- Rate-limit inconsistency between RateLimit.sol and ExitLimitUtils.sol leading to mis-sized limits
- $O(n^2)$ unsorted top-up loop — gas griefing / inefficiency

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	1	1	1
High	-	-	-
Medium	5	5	5
Low	7	9	3
Informational	10	10	7
Total	23	23	16

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
C-01	An attacker can DOS the accounting oracle by depositing extra ETH (e.g., 1 ETH) to a Lido validator from a non-Lido address, causing the reported <code>clPendingBalance</code> to exceed the expected <code>preCLPendingBalance</code> + deposits and reverting the sanity check	Critical	Fixed
M-01	Possible revert in sanity checks due to <code>`_lastVaultBalanceAfterTransfer`</code> value set during migration	Medium	Fixed
M-02	An attacker can permanently block a Lido validator's consolidation queue by initiating a consolidation from a non-Lido validator and then getting slashed to cancel it (inflating the target's pending balance above the threshold)	Medium	Fixed
M-03	In <code>finalizeUpgrade_v4</code> , the <code>oracle.getLastProcessingRefSlot() == 0</code> path incorrectly bypasses the check that the report for the current epoch was submitted correctly, which could result in deposited ETH being lost.	Medium	Fixed
M-04	CL balance increase can mistakenly be flagged as decrease, causing revert	Medium	Fixed

M-05	Decrease sanity check does not account for deposits which allows deposit value to be dropped from accounting	Medium	Fixed
L-01	TWG can be DoSed due to ability to trigger the same withdrawals multiple times	Low	Acknowledged
L-02	Potential to top up a validator that is not yet deposited	Low	Acknowledged
L-03	_calculateTotalProtocolFeeShares does not use post-rebase share rate	Low	Acknowledged
L-04	Deposits and Top ups that use nearly the full deposit reserve and unreserved ETH can be grieved	Low	Acknowledged
L-05	Potentially unsafe maxPositiveTokenRebase parameter supplied to sanity checker	Low	Fixed
L-06	Rebase smoothing is malleable	Low	Acknowledged
L-07	APR safety cap calculation does not take into account activated balance	Low	Fixed
I-01	DepositSecurityModule.canDeposit() should reflect emergency setting quota larger than guardians.length	Informational	Fixed
I-02	Consolidations should check deposit pause edge case	Informational	Acknowledged

I-03	TopupGateway validator check is inefficient and does not require sorting	Informational	Fixed
I-04	Inconsistencies in rate limit mechanisms	Informational	Acknowledged
I-05	Missing return value description for `depositedForCurrentReport`	Informational	Fixed
I-06	Incorrect reference to `obtainDepositData` function	Informational	Fixed
I-07	consolidationEthAmountPerDayLimit discrepancy in docs	Informational	Fixed
I-08	Incorrect reference to variable clPendingValidatorsBalance	Informational	Fixed
I-09	Incorrect calculation in docs for clRewards	Informational	Acknowledged
I-10	Incorrect comment for CL_VALIDATORS_BALANCE_AND_CL_PENDING_BALANCE_POSITION	Informational	Fixed

Critical Severity Issues

C-01: DoS of Accounting Oracle report through direct donation

Severity: Critical	Impact: High	Likelihood: Medium
Files: core/contracts/0.8.9/sanity_checks/OracleReportSanityChecker.sol	Status: Fixed	

Description:

An attacker can DOS the accounting oracle by depositing extra ETH (e.g., 1 ETH) to a Lido validator from a non-Lido address, causing the reported `clPendingBalance` to exceed the expected `preCLPendingBalance` + deposits, causing the sanity check to revert.

The following check imposes a strict condition that the `postCLPendingBalance` must be less than or equal to the previous pending balance plus deposits. Therefore, a minimum donation of 1 ETH can cause the report to revert, delaying withdrawals in the process.

JavaScript

```
if (_checkParams.postCLPendingBalance > pendingBalanceWithDeposits) {  
    revert IncorrectTotalPendingBalance(pendingBalanceWithDeposits,  
    _checkParams.postCLPendingBalance);  
}
```

Customer Response:

Fixed in [PR #1821](#).

Fix Review:

We confirm that the fix disallows causing a revert in the accounting report process through direct donation of small amounts, e.g. 1 ETH.

Medium Severity Issues

M-01: Possible revert in sanity checks due to ``_lastVaultBalanceAfterTransfer`` value set during migration

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/0.8.9/sanity_checks/OracleReportSanityChecker.sol	Status: Fixed	

Description:

The `_lastVaultBalanceAfterTransfer` variable is set in the `OracleReportSanityChecker` contract at time of migration. This variable is intended to account for the ETH balance in the withdrawal vault leftover after the previous accounting oracle report to accurately capture the amount of ETH withdrawn from the consensus layer in the current report period.

In the migration function `migrateBaselineSnapshot`, the `_lastVaultBalanceAfterTransfer` value is set to `LIDO_LOCATOR.withdrawalVault().balance`.

However, the current ETH balance of the withdrawal vault may already incorporate some amount of withdrawals in the current frame that need to be accounted for. If the recent withdrawals are not properly accounted for, the `_checkCLBalanceDecrease` sanity check may revert, causing the accounting oracle report process to revert.

Recommendations:

Set `_lastVaultBalanceAfterTransfer` to the proper ETH balance of the withdrawal vault at the time immediately following the previous report. This method will allow all withdrawals that occurred in the current frame to be accounted for, while not over-accounting for the latent ETH balance that already existed in the withdrawal vault pre-migration.

Customer Response:

Fixed in commit [0a46caa7](#).

Fix Review:

We confirm that the balance of the withdrawal vault is properly accounted for in the seeded report data that affect the decrease sanity checks.

M-02: Non-Lido Consolidation Can Grief Lido Validator Selection

Severity: Medium	Impact: Medium	Likelihood: Low
Files: DepositorBot Offchain Code	Status: Fixed	

Description:

An attacker can initiate a consolidation from their own non-Lido validator into a Lido validator that is about to receive a legitimate consolidation. Because the pending incoming balance would push the target validator above the 2048 ETH limit, Lido's off-chain consolidation logic may skip that validator.

The attacker can then get the source validator slashed before the consolidation completes, causing the consolidation to be cancelled. This lets the attacker repeatedly force Lido to skip otherwise valid validators, as long as they are willing to pay the request cost and absorb a small slash. Since the source validator is not controlled by Lido and has no Lido bond, there is no practical way for the protocol to penalize the attacker.

A simple mitigation is for the off-chain consolidation logic to ignore pending consolidations that originate from non-Lido withdrawal credentials. If the external consolidation succeeds, the extra ETH is just a donation that can later be swept. If it is cancelled, it should not affect Lido's own consolidation decisions.

Customer Response:

Depositor bot has been updated to mitigate this issue and the documentation has been updated.

Fix Review:

Lido has mitigated this possibility by updating the depositor bot which runs off-chain.

M-03: Potential loss of deposits in fresh deploy

Severity: Medium	Impact: High	Likelihood: Low
Files: core/contracts/0.4.9/Lido.sol	Status: Fixed	

Description:

In `finalizeUpgrade_v4`, execution is allowed when no previous reports have been submitted via the clause:

JavaScript

```
require(mainDataSubmitted || oracle.getLastProcessingRefSlot() == 0,  
"NO_REPORT");
```

This path incorrectly bypasses the check that the report for the current epoch was submitted correctly which could result in deposited ETH being lost.

This code path is only vulnerable on fresh deploys of the Lido protocol, therefore it is not vulnerable in the anticipated upgrade.

Customer Response:

Fixed in [PR #1821](#).

Fix Review:

Lido has fixed this potential issue and now requires that at least one report has been submitted prior to upgrade.

M-04: CL balance increase can mistakenly be flagged as decrease, causing revert

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/0.8.9/sanity_checks/OracleReportSanityChecker.sol	Status: Fixed	

Description:

Checks prior to window calcs verify that CL balance does in fact decrease from last report to current report, however, then the **postCLBalance** is compared against the baseline value from the report 36 days ago. So a possible scenario looks like:

Report 0 -> 1_000_000

Report 1 -> 1_100_000 (increase)

Report 2 -> 1_200_000 (increase)

Report 3 -> 1_300_000 (increase)

Report 4 -> 1_200_000 (decrease of 100_000 from previous report)

Then the **actualCLBalanceDiff** is calculated as an absolute value:

```
uint256 baselineBalance = reportData[baselineIndex].clBalance;
actualCLBalanceDiff = baselineBalance > _postCLBalance
    ? baselineBalance - _postCLBalance
    : _postCLBalance - baselineBalance;
```

Therefore **actualCLBalanceDiff = 1_200_000 - 1_000_000 = 200_000**

The total ETH in the system has increased by 200_000 but it is flagged as a 200_000 ETH loss which may trigger the decrease sanity check if it exceeds 3.6% in the previous 36-day period.

Customer Response:

Fixed in [PR #1821](#).

Fix Review:

We confirm that the decrease sanity check has been updated so that the above case will not result in a revert. The decrease sanity check has been converted to calculate the total ETH in the Lido system and works as intended.

M-05: Decrease sanity check does not account for deposits which allows deposit value to be dropped from accounting

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/0.8.9/sanity_checks/OracleReportSanityChecker.sol	Status: Fixed	

Description:

In `OracleReportSanityChecker._calcWindowDiff`, the absolute difference of the CL balance decrease is calculated against the raw historical baseline: `actualCLBalanceDiff = abs(baselineBalance - _postCLBalance)`.

While the maximum allowed decrease (`maxAllowedCLBalanceDiff`) appropriately accounts for total deposits and withdrawals over the window, the `actualCLBalanceDiff` does not.

If the protocol deposits funds to CL and the validator set concurrently suffers a massive slashing event of the same amount, `_postCLBalance` will equal `baselineBalance` causing `actualCLBalanceDiff` to be 0. This hides the economic loss from the sanity checker and effectively drops the value of the deposits from Lido's accounting.

Recommendations:

Calculate the gain or loss based on the total ETH in system, not solely `clBalance` and `pendingCLBalance`.

Customer Response:

Fixed in [PR #1821](#).

Fix Review:

The decrease sanity check has been converted to calculate the total ETH in the Lido system and works as intended.

Low Severity Issues

L-01: TWG can be DoSed due to ability to trigger the same withdrawals multiple times

Severity: Low	Impact: Medium	Likelihood: Low
Files: core/contracts/0.8.9/TriggerableWithdrawalsGateway.sol	Status: Acknowledged	

Description

The TriggerableWithdrawalsGateway contract can be DoSed because each withdrawal request can be triggered an infinite number of times permissionlessly. Once the exit data has been submitted, any actor can trigger the withdrawal multiple times, consuming the rate limit multiple times.

In practice, this will likely not cause a serious impact as the rate limit is a last line of defense, while verification of the exit requests is performed upstream.

Customer Response:

We assume that off-chain monitoring will detect such cases and corresponding changes to the TWG limit can be made by the DAO to counteract the attack.

Fix Review:

We agree that Lido maintains the ability to update or increase the rate limit parameters to mitigate this issue.

L-02: Potential to top up a validator that is not yet deposited

Severity: Low	Impact: Low	Likelihood: Low
Files: core/contracts/0.8.25/Topup Gateway.sol	Status: Acknowledged	

Description:

A top-up can be allocated to a pre-deposit validator (with 1 ETH) that hasn't been activated yet (i.e., its `activationEpoch` is `FAR_FUTURE_EPOCH`). Based on the implementation of the Topup Gateway, it should be disallowed to top-up a validator that has not yet been activated.

We recommend adding a check to ensure that only deposited validators can be the target of a top up.

Customer Response:

Acknowledged.

L-03: `_calculateTotalProtocolFeeShares` does not use post-rebase share rate

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/0.8.9/Accounting.sol	Status: Acknowledged	

Description:

When calculating the total protocol fee shares, it states **we are calculating the share rate equal to the post-rebase share rate**. However, the function calculates the shares to mint as:

```
sharesToMintAsFees = (feeEther * _internalSharesBeforeFees) /  
(_update.postInternalEther - feeEther);
```

`_internalSharesBeforeFees` does not encapsulate `badDebtToInternalize`. This value increases the post-report internal shares later in the execution.

Therefore it seems the protocol fee shares calculation doesn't truly use the post-rebase share rate in case of bad debt.

Customer Response:

Acknowledged.

Fix Review:

This calculation is not new to the updated StakingRouter scope.

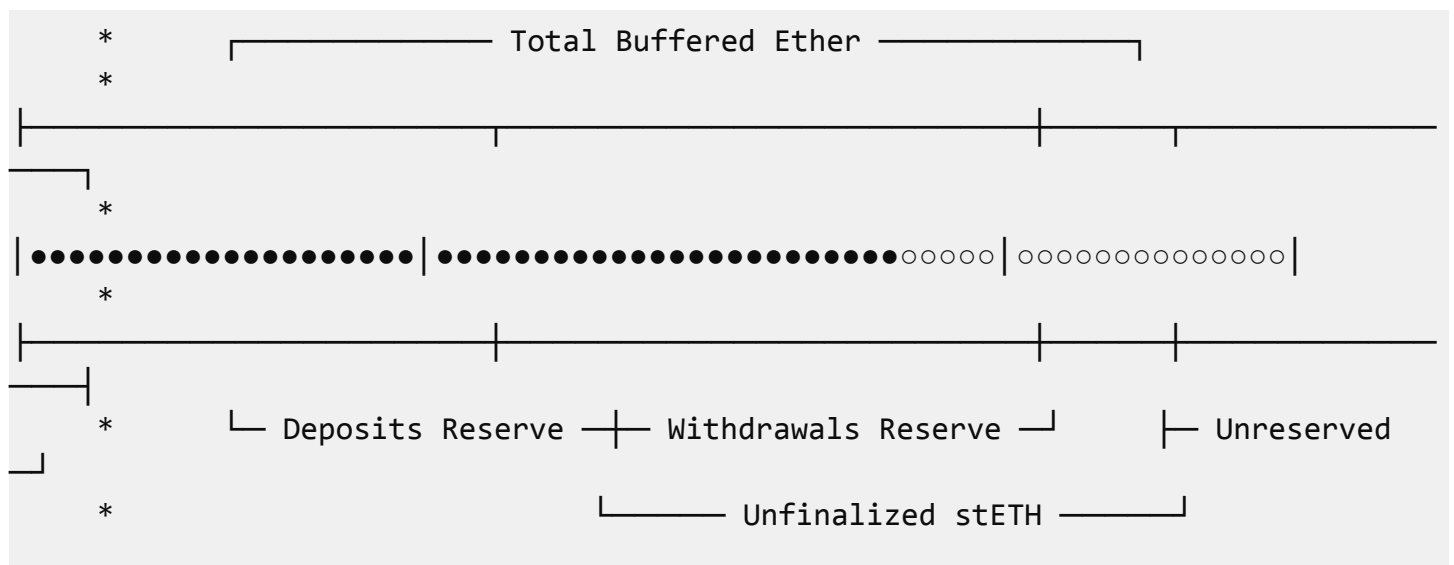
L-04: Deposits and Top ups that use nearly the full deposit reserve and unreserved ETH can be grieved

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/0.4.24/lido.sol	Status: Acknowledged	

Description:

There is a minor grieving vector that allows a 3rd party to block deposits and top up operations that aim to use nearly the full deposit reserve and unreserved allocation.

The reserve system is as follows:



The protocol is able to deposit the entire deposit reserve and any unreserved ETH available in the buffer.

However, if Lido attempts to deposit the full deposit reserve and all or most unreserved ETH, if anyone creates a new withdrawal request prior to the deposit/top up transaction, the transaction will revert because the withdrawals reserve increases while simultaneously decreasing the unreserved allocation.

The total depositable ether can be reduced incrementally to DoS all calls to deposit or top up that are near the max deposit reserve + unreserved allocation.

Customer Response:

Acknowledged.

L-05: Potentially unsafe maxPositiveTokenRebase parameter supplied to sanity checker

Severity: Low	Impact: Low	Likelihood: Low
Files: https://hackmd.io/pcXEmYWRRROWZ_739zZ1PGA?view=&text=13077%3A1683%3A0%3A1779967801%3A16R8pP	Status: Fixed	

Description:

The documentation states that the supplied `maxPositiveTokenRebase` will be 5000000 or 5e6. However, the comment above the getter function for this value states:

```
/// the recommended sane values are from 0.05% to 0.1%.
```

5e6 equates to 0.5% which is above the stated sane values.

Recommendations:

Either reconsider the supplied value or update the comment if 0.5% is acceptable.

Customer Response:

The 5e6 value is indeed not relevant for mainnet, though it may be relevant for testnets. The current value there is 750000 (0.075%).

Fix Review:

We agree that a value of 750000 is an acceptable value for `maxPositiveTokenRebase`.

L-06: Rebase smoothing is malleable

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/0.8.9/Accounting..sol	Status: Acknowledged	

Description:

The comment at Accounting.sol:184 says `// Limit the rebase to avoid oracle frontrunning`, but that doesn't actually hold. `Accounting._snapshotPreReportState` reads `LIDO.getTotalPooledEther()` at execution time, so any `submit()` landing in the same block as `handleOracleReport` is already baked into `pre.totalPooledEther`. T

The inflated value feeds `PositiveTokenRebaseLimiter.initLimiterState`, which sets `maxTotalPooledEther = preTPE * (1 + MPR)`. The attacker's own submission widens the cap by `X * MPR`.

Recommendations:

Since the Lido currently holds a large amount of ETH and the staking limit is relatively small, front-running the rebase is not likely to occur in practice. However, the comment is slightly misleading and might be removed or reworded.

Customer Response:

Acknowledged. This scenario seems to work only with a very small stake size during a fresh deployment, which is not our case. Given Lido's current stake size, such an attack is not economically viable.

Fix Review:

We agree with the above response.

L-07: APR safety cap calculation does not take into account activated balance

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/0.8.9/sanity_checks/OracleReportSanityChecker.sol	Status: Fixed	

Description:

In `_checkCLPendingBalanceAndCalculateActivatedBalanceWithGap()`, the `activatedBalanceWithGap` is calculated as:

```
activatedBalance + (_preCLValidatorsBalance * _annualBalanceIncreaseMultiplier) /  
ANNUAL_BALANCE_INCREASE_DENOMINATOR;
```

This calculates the gift and apr safety cap based on the `_preCLValidatorsBalance`. But technically, the activated balance could have been activated at the beginning of the frame, e.g. in the first slot after the previous ref slot, meaning the apr safety cap would be underestimating the ETH that was actually earning the APR, making the check a little tighter than expected.

In an extreme case such as if the oracle reports have been skipped for several days, the APR safety cap is calculated based on the `_preCLValidatorsBalance` from multiple days ago while deposits would have been made and activated the whole time.

Recommendations:

Add the activated balance to the gift and apr safety cap calculation.

Customer Response:

Fixed in [PR #1821](#).

Fix Review:

We confirm that the activated balance is now included in the safety cap calculation.

Informational Severity Issues

I-01: `DepositSecurityModule.canDeposit()` should reflect emergency setting quota larger than `guardians.length`

Files:

`core/contracts/0.8.9/DepositSecurityModule.sol`

Description:

The `DepositSecurityModule` includes a `canDeposit()` view function that returns if the conditions for deposit are met. However, Lido maintains an emergency methodology to pause deposits by setting the quorum larger than the length of set guardians, effectively blocking the signature verification from succeeding. If Lido performs this maneuver, `canDeposit()` will still return `true`.

Therefore, it is recommended to add a check for this emergency case when returning `true` or `false` from `canDeposit()`.

Customer Response:

Fixed in commit [1a6d00cc](#).

Fix Review:

We verify that the fix is in place that restricts deposits when the the quorum is set larger than the guardian set length.

I-02: Consolidations should check deposit pause edge case

Files:

core/contracts/0.8.25/Consolidations/ConsolidationGateway.sol

Description:

Consolidations can only occur through the `ConsolidationsGateway` contract if the `_checkConsolidationsPreconditions` function returns true. This function checks that `DepositSecurityModule.isDepositsPaused` returns `false`.

However, as raised in I-01, Lido has an emergency mechanism for pausing deposits by setting the quota larger than the length of set signers.

Therefore, it is recommended to check for this emergency case when checking consolidation preconditions as well.

Customer Response:

Acknowledged.

I-03: TopupGateway validator check is inefficient and does not require sorting

Files:

core/contracts/0.8.25/Consolidations/TopupGateway.sol

Description:

The TopUp flow performs a nested for loop resulting in n^2 growth. We recommend ensuring that the input validators are sorted, removing the need for the nested for loops.

Customer Response:

Fixed in commit [85be813a](#).

Fix Review:

Lido has fixed this issue by removing the nested for loops and require ascending order of validators.

I-04: Inconsistencies in rate limit mechanisms

Description:

ConsolidationGateway and ValidatorsExitBus use the same implementation of RateLimit.sol, while TWG uses a very similar but modified version ExitLimitUtils.sol.

A notable difference is seen when updating the max limits.

In `RateLimit.sol`, if the new max limit is less than the remaining amount, simply set remaining amount to new max limit. As previously discussed, this path does not checkpoint the rate limit state so replenishment is ignored.

- Remaining = 5
- New max limit = 3
- Remaining becomes 3

`ExitLimitUtils.sol` uses a different approach where replenishment is achieved and the new remaining amount is set to new max limit minus the current consumption.

- Prev max limit = 7
- Remaining = 5
- Current used = $(7-5) = 2$
- New max limit = 3
- Remaining becomes $(3-2) = 1$

Customer Response:

Acknowledged.

I-O5: Missing return value description for `depositedForCurrentReport`

Files:

core/contracts/O.4.24/Lido.sol

Description:

The `Lido.getBalanceStats` function documents the first three return values but omits the fourth.

Customer Response:

Fixed in commit [c68d9aa3](#).

Fix Review:

We confirm all return values are documented.

I-06: Incorrect reference to `obtainDepositData` function

Description:

`BeaconChainDepositor.makeBeaconChainTopUp` contains the comment:

```
// obtainDepositData can return 0 amount for some keys
```

However, `obtainDepositData` is not called in this flow. Instead, `allocate()` is called on the module side.

Customer Response:

Fixed in commit [9bb84c44](#).

Fix Review:

The comment has been updated.

I-07: consolidationEthAmountPerDayLimit discrepancy in docs

Description:

Constructor argument states 93375 but description states 91800.

Customer Response:

Updated documentation.

Fix Review:

Confirmed, the values now match.

I-08: Incorrect reference to variable `clPendingValidatorsBalance`

Description:

The docs reference `clPendingValidatorsBalance` variable which does not exist in the codebase.

Customer Response:

Reference has been removed.

Fix Review:

Confirmed.

I-09: Incorrect calculation in docs for clRewards

Description:

In the documentation, the clRewards calculation is stated as `clRewards = (report.clValidatorsBalance + report.clPendingBalance) + update.withdrawalsVaultTransfer - principalClBalance`

This omits the `elRewardsVaultTransfer` value.

Customer Response:

Acknowledged.

I-10: Incorrect comment for state variable

Description:

The comment states that the `CL_VALIDATORS_BALANCE_AND_CL_PENDING_BALANCE_POSITION` holds the `deposited balance since last report` value however this is not true.

```
/// @dev CL validators balance and deposited balance since last report
/// |----- 128 bit -----|----- 128 bit -----|
/// | CL validators balance   | CL pending balance   |
/// keccak256("lido.Lido.clValidatorsBalanceAndClPendingBalance");
bytes32 internal constant
CL_VALIDATORS_BALANCE_AND_CL_PENDING_BALANCE_POSITION =
    0x096e465397f38e659238ccd5d5a2c434ced54a63fd8d694045bfb058ab9d8112
```

Recommendations:

Update the comment to specify the actual value tracked, `clPendingBalance`

Customer Response:

Fixed in [PR #1821](#).

Fix Review:

Comment was fixed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.